Investment Advisor

Agentic AI system — implementation-focused technical architecture

- Python • LangGraph • LangChain • FastAPI • PostgreSQL • pgvector
- Specialized agents for query understanding, customer context, knowledge retrieval, risk, portfolio analysis and compliance
- Scope: local development; no production deployment or regulatory approval claimed

Project objective

UK-focused investment education and controlled guidance

- Combine customer financial context with retrieved investment and policy knowledge
- Distinguish educational queries from customer-specific investment requests
- Escalate uncertain suitability or compliance cases using a needs_review outcome
- No automated investment execution or completed human-review queue

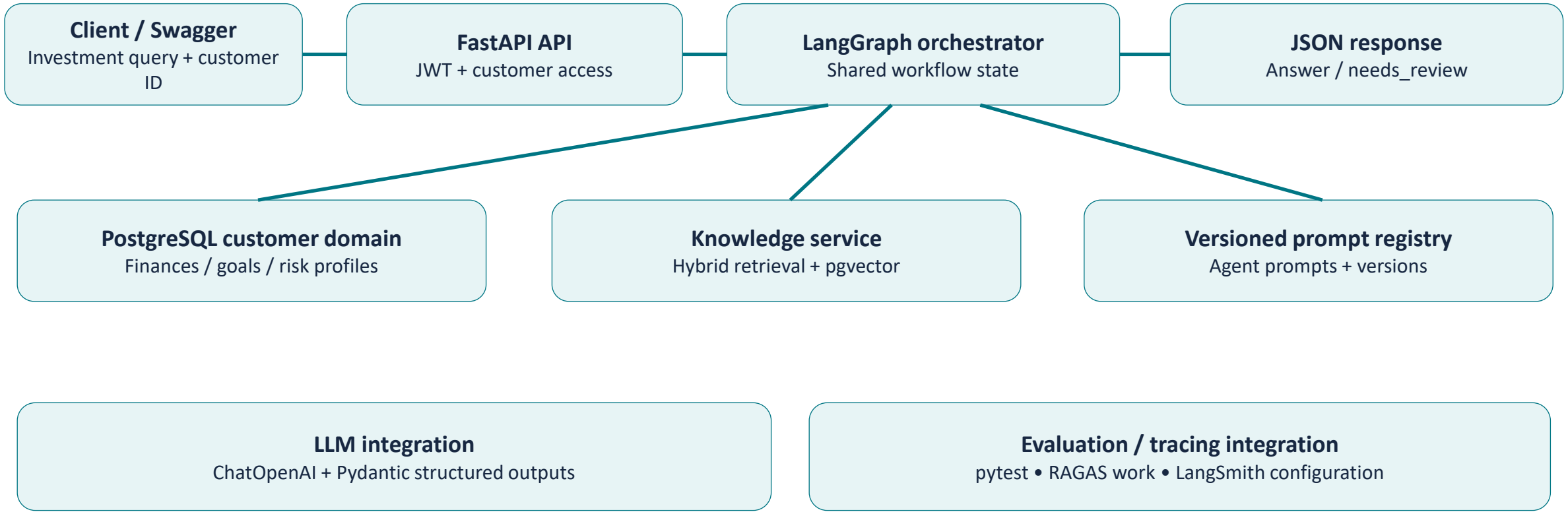
Application architecture

Client → secured API → LangGraph → PostgreSQL and RAG

- FastAPI exposes login, current-user, health and investment-analysis routes
- JWT authentication and per-customer authorization precede agent execution
- LangGraph coordinates the query, customer, knowledge, risk, portfolio and compliance steps
- PostgreSQL stores customer data, authentication mappings, knowledge and prompt versions

01 | Application architecture — end-to-end

Request path, agent orchestration, data services and response handling



Status note: components and integration code were developed locally; full end-to-end validation, RAGAS scoring and live LangSmith trace delivery have not been confirmed.

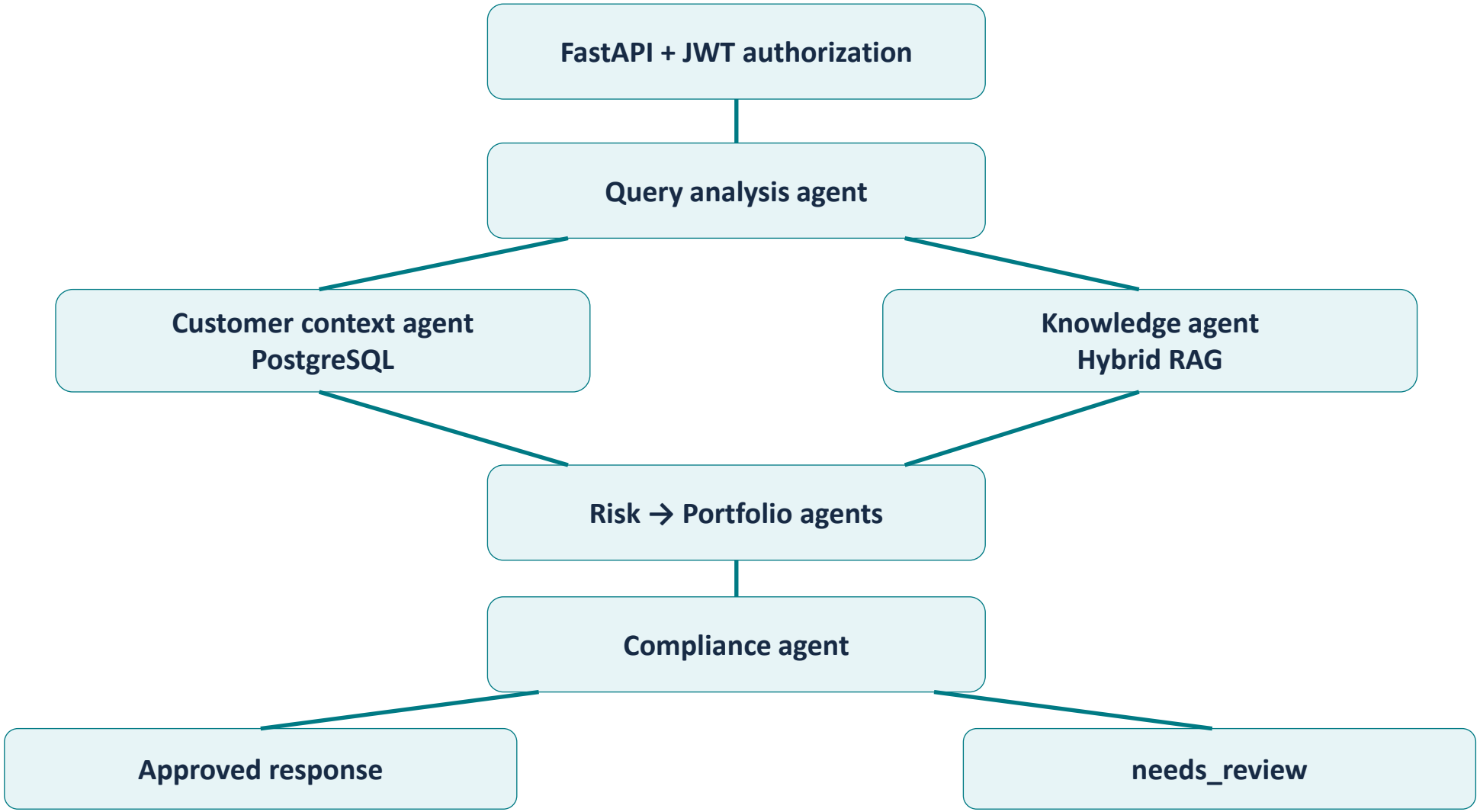
LangGraph workflow

Implemented graph structure and a previously tested escalation

- Query analysis → parallel customer and knowledge retrieval
- Risk analysis → portfolio analysis → compliance validation
- Approved response or needs_review result
- A prior local test returned needs_review for aggressive tolerance versus moderate capacity
- Current graph after subsequent refactoring has not been independently revalidated

Agentic AI — LangGraph workflow

Implemented graph structure • review branch is a status, not a persisted human-review service



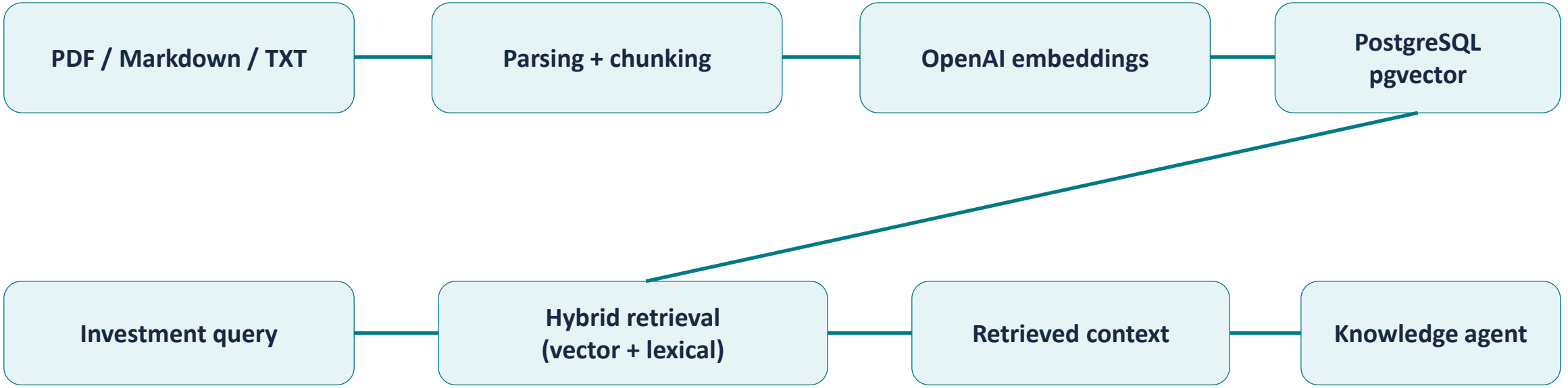
RAG implementation

Development knowledge base and retrieval pipeline

- Sample PDF, Markdown and text documents: risk, compliance, benefits, investment education and FCA-related concepts
- Parsing and chunking → OpenAI text-embedding-3-small → PostgreSQL pgvector
- Hybrid lexical/vector retrieval and reranking components developed
- Sample policy content is not verified live FCA guidance; no benchmark scores claimed

RAG — ingestion and retrieval architecture

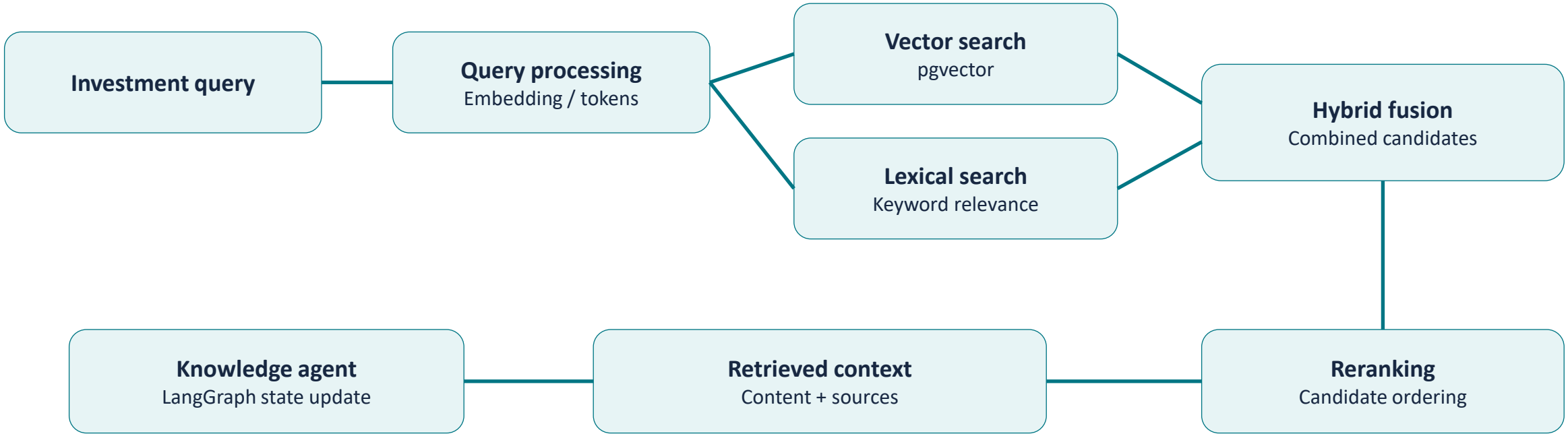
Development knowledge base: sample investment education, risk and compliance documents



Scope: ingestion and hybrid-retrieval components developed locally. Source policies are illustrative; retrieval quality and reranking have not been independently benchmarked.

04 | RAG architecture — online retrieval

Semantic + lexical retrieval, context assembly and agent handoff



Boundary: retrieval and reranking components were developed, but retrieval accuracy, reranker lift and end-to-end benchmarks have not been independently established.

Data architecture

Normalized customer domain, authentication and prompt registry

- Customer tables: employment, finances, liabilities, goals, investments, preferences and risk profiles

- api_users stores hashed credentials; api_user_customers maps permitted customer access
- agent_prompts stores agent-specific system prompts and active versions
- CUST-003 sample data was used to exercise a risk-mismatch scenario

FastAPI and JWT

Local API implementation and security boundary

- GET /health; POST /api/v1/auth/login; GET /api/v1/auth/me
- POST /api/v1/investment/analyze invokes the graph after customer access validation
- Argon2 password hashing (pwdlib), PyJWT and async SQLAlchemy
- Authentication tables created successfully; complete login-to-analysis test not yet confirmed

Prompt management

Versioned system instructions and structured outputs

- Shared asynchronous prompt loader reads active prompt records from PostgreSQL
- Prompt delimiters: <role>, <objective>, <instructions>
- Query agent uses ChatOpenAI with Pydantic structured output
- Prompt ID/version recorded in agent state; no completed v1-versus-v2 benchmark claimed

Evaluation and tracing

Distinguish implemented starter code from verified results

- pytest: banking-rule and API test examples developed
- RAGAS: evaluation integration attempted; dependency import conflict remains unresolved
- LangSmith: tracing
- No confirmed production traces, quality benchmark, or regulatory validation claimed

Current implementation status

Evidence-based summary

- Developed locally: customer database, multi-agent graph, RAG components and prompt registry
- Authentication tables created; FastAPI/JWT endpoints implemented in code
- Earlier graph test passed for needs_review; later module refactoring required fixes
- Not completed: production deployment, CI/CD, persistent reviewer queue, end-to-end verification

Future Improvements (if required) and Production Roadmap

1.Security: RBAC, PII masking, prompt injection protection, rate limiting and token management.

2.Agent & Tool Security: Tool permissions, input validation, execution limits and access control.

3.RAG Enhancement: Document versioning, source verification, access control and retrieval optimization.

4.Financial Compliance: Suitability engine, FCA policy enforcement, risk assessment and explainable decisions.

5.Human-in-the-Loop: Approval workflows, adviser dashboard and LangGraph checkpointing.

6.Reliability & Performance: Retry mechanisms, caching, error handling, latency and token-cost optimization.

7.Testing & Evaluation: RAGAS, LangSmith, security testing, regression testing and load testing.

8.Monitoring & Auditing: Centralized logging, agent tracing, audit trails and automated alerts.

9.GCP Deployment: Docker, Cloud Run, Cloud SQL, Secret Manager and Redis.

10.Scalability: Auto-scaling, asynchronous processing, database optimization and disaster recovery.

Next steps: GCP deployment + CI/CD

Future work only — not implemented

- Target Google Cloud Platform (GCP): Cloud Run for FastAPI + LangGraph
- Stabilize local workflow, authentication and regression tests; pin dependencies
- Build Docker image; publish to Artifact Registry; use Secret Manager and Cloud SQL PostgreSQL
- GitHub Actions CI: lint, pytest, dependency checks, Docker build; Workload Identity Federation
- GitHub Actions CD: deploy to Cloud Run, approvals, rollback and monitoring